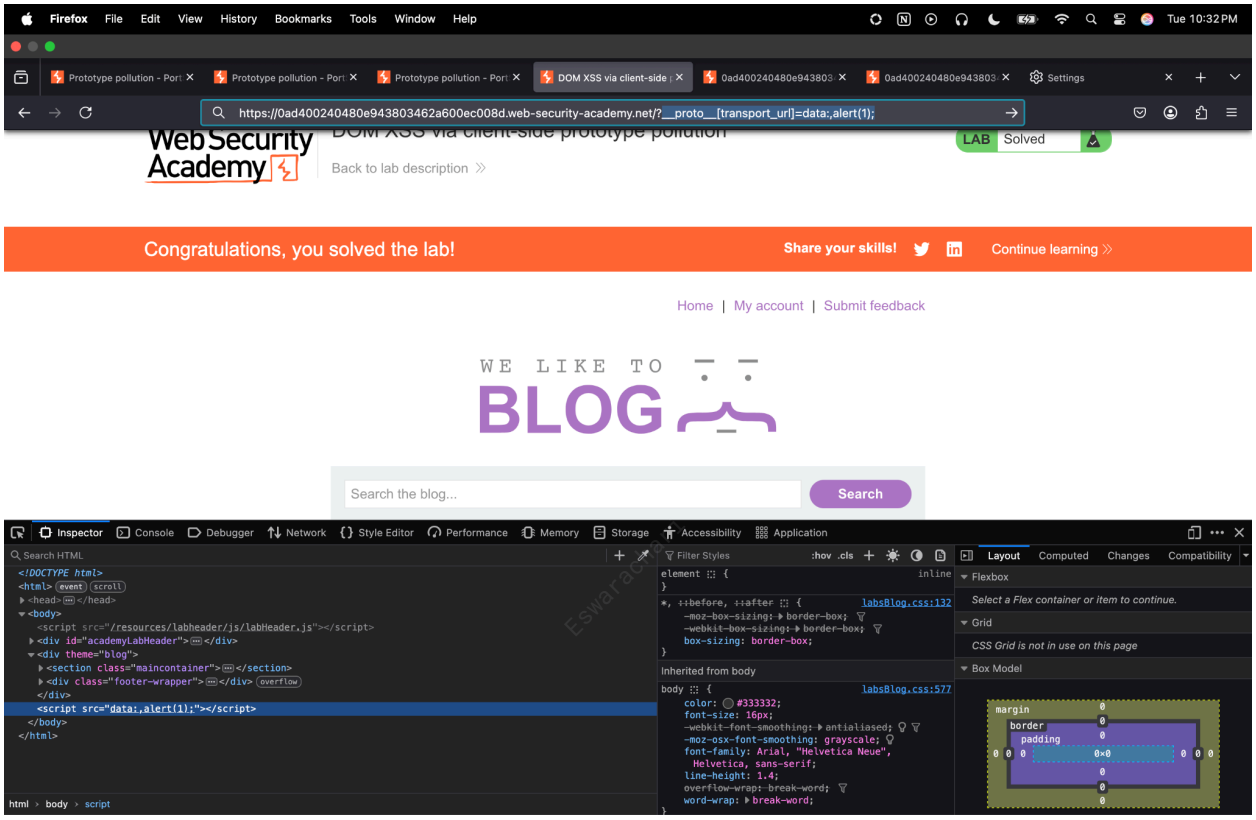
1.DOM XSS via client-side prototype pollution

Lab Name: DOM XSS via client-side prototype pollution

Vulnerability Type: DOM-Based Cross-Site Scripting (DOM XSS)



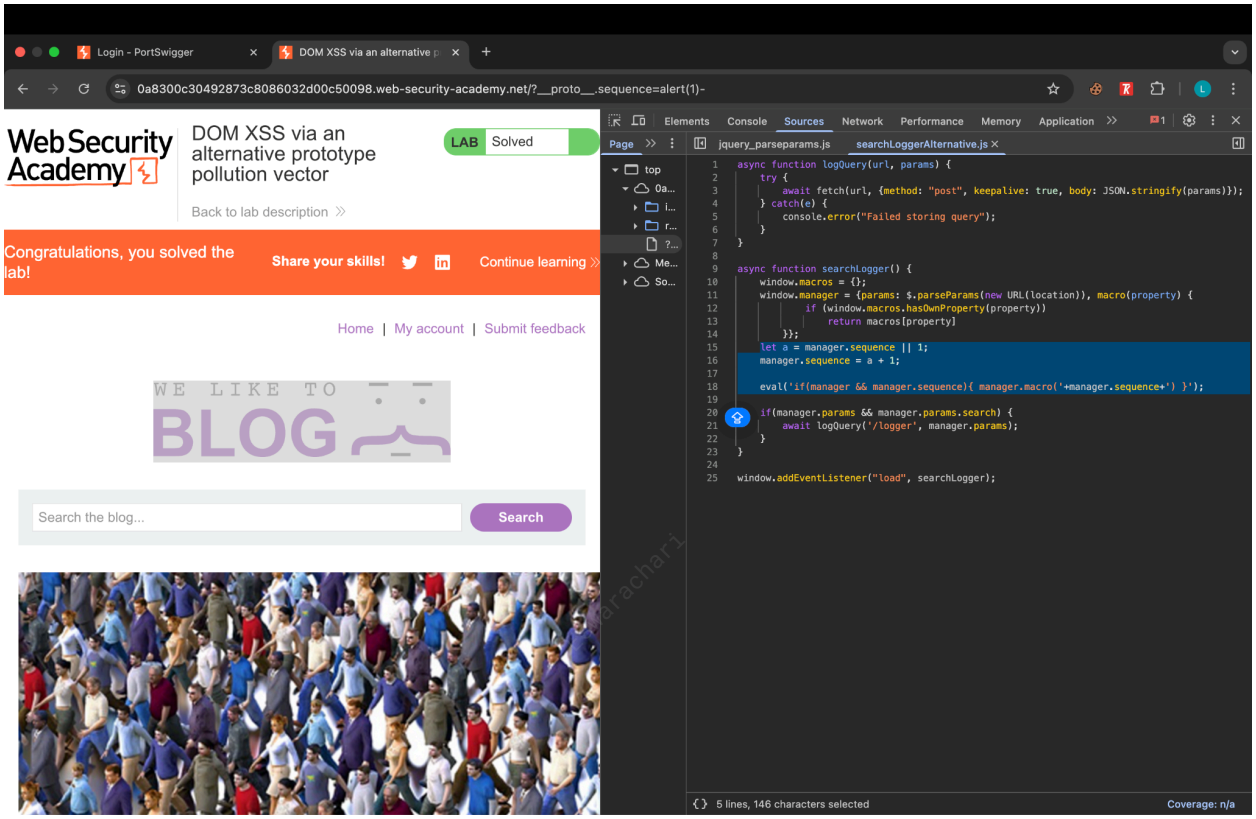
Summary: This lab demonstrates how client-side prototype pollution can be exploited to inject and execute malicious JavaScript code. By manipulating the `_proto_` property in the URL parameters, it's possible to overwrite built-in JavaScript object properties. In this case, I injected `alert(1)` which caused a popup box to appear, proving the execution of arbitrary JavaScript. This type of vulnerability can be used to steal user data, hijack sessions, or deface websites.

Screenshot Context: The screenshot shows the URL with the `_proto_[transport_url]` parameter set to `data:,alert(1);` which injects and executes the JavaScript code.

2. DOM XSS via an alternative prototype pollution vector

Lab Name: DOM XSS via an alternative prototype pollution vector

Vulnerability Type: DOM-Based Cross-Site Scripting (DOM XSS)



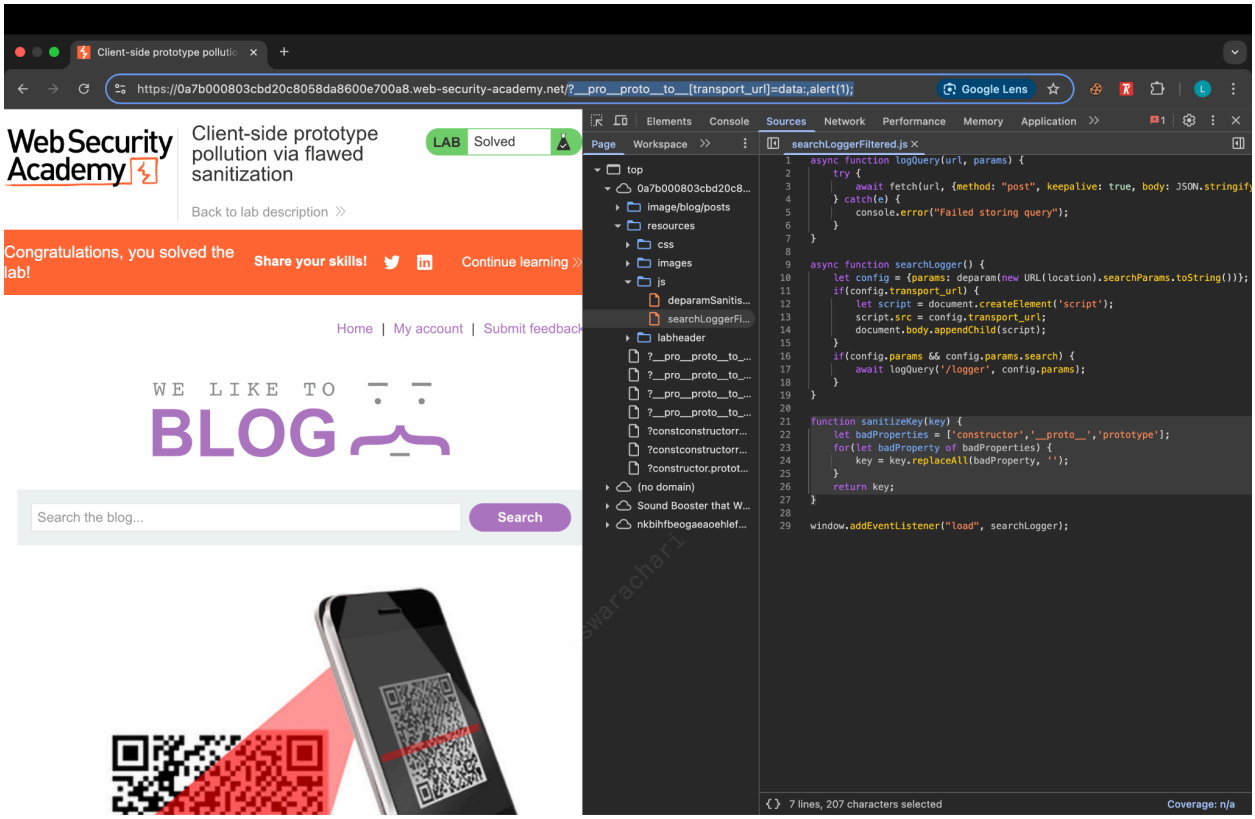
Summary: This lab explores an alternative method of exploiting prototype pollution to achieve DOM XSS. The application uses a library function (`$.parseParams`) to parse URL parameters, and this function is vulnerable to prototype pollution. By carefully crafting the URL, I was able to inject malicious JavaScript that executed within the page, resulting in XSS. This highlights that even different parsing methods can be a source of prototype pollution vulnerabilities.

Screenshot Context: The screenshot shows the URL with the `_proto_sequence` parameter set to `alert(1)`, demonstrating the prototype pollution exploit

3.Client-side prototype pollution via flawed sanitization

Lab Name: Client-side prototype pollution via flawed sanitization

Vulnerability Type: Client-Side Prototype Pollution



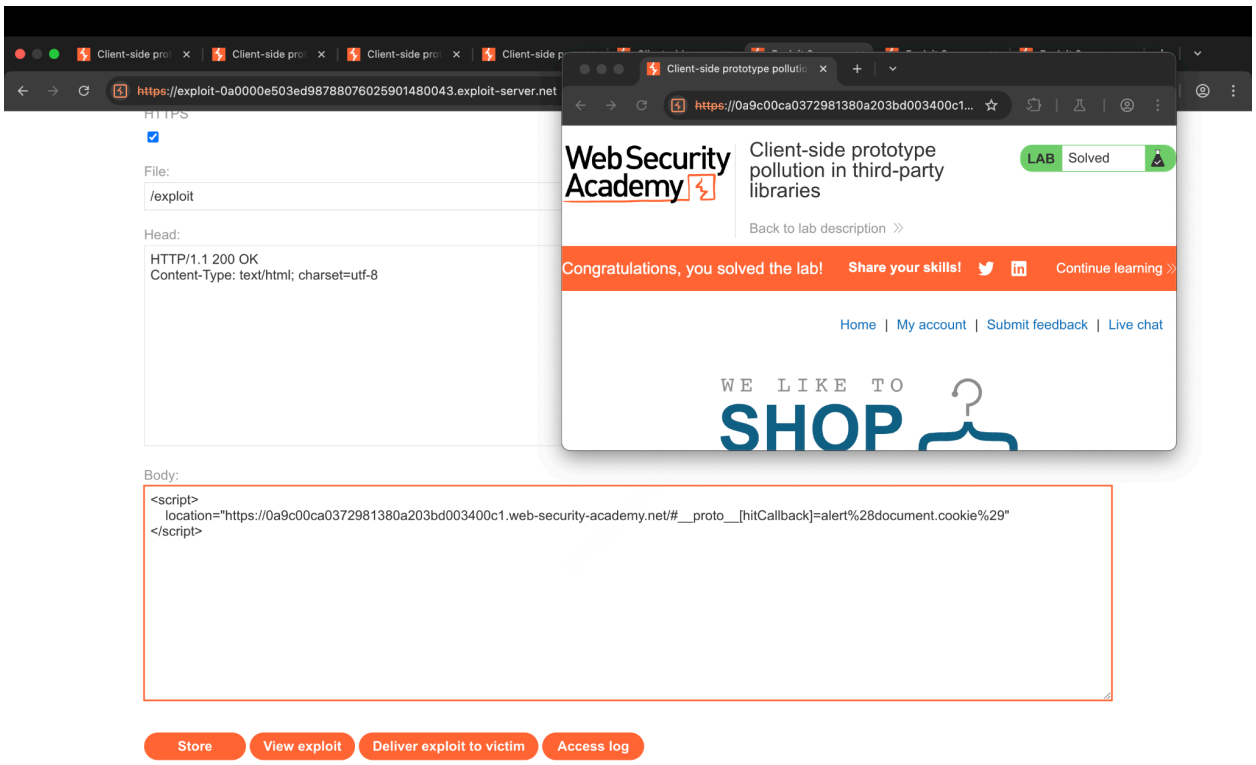
Summary: This lab focuses on bypassing flawed sanitization mechanisms in a client-side application to exploit prototype pollution. The application attempts to sanitize potentially dangerous properties but does so inadequately. By exploiting this flaw, I was able to inject malicious JavaScript through the `transport_url` parameter, leading to the execution of arbitrary code. This shows that even with sanitization in place, vulnerabilities can still exist if it's not implemented correctly.

Screenshot Context: The screenshot shows the URL with the `_pro_proto_to_[transport_url]` parameter used to inject JavaScript.

4. Client-side prototype pollution in third-party libraries

Lab Name: Client-side prototype pollution in third-party libraries

Vulnerability Type: Client-Side Prototype Pollution



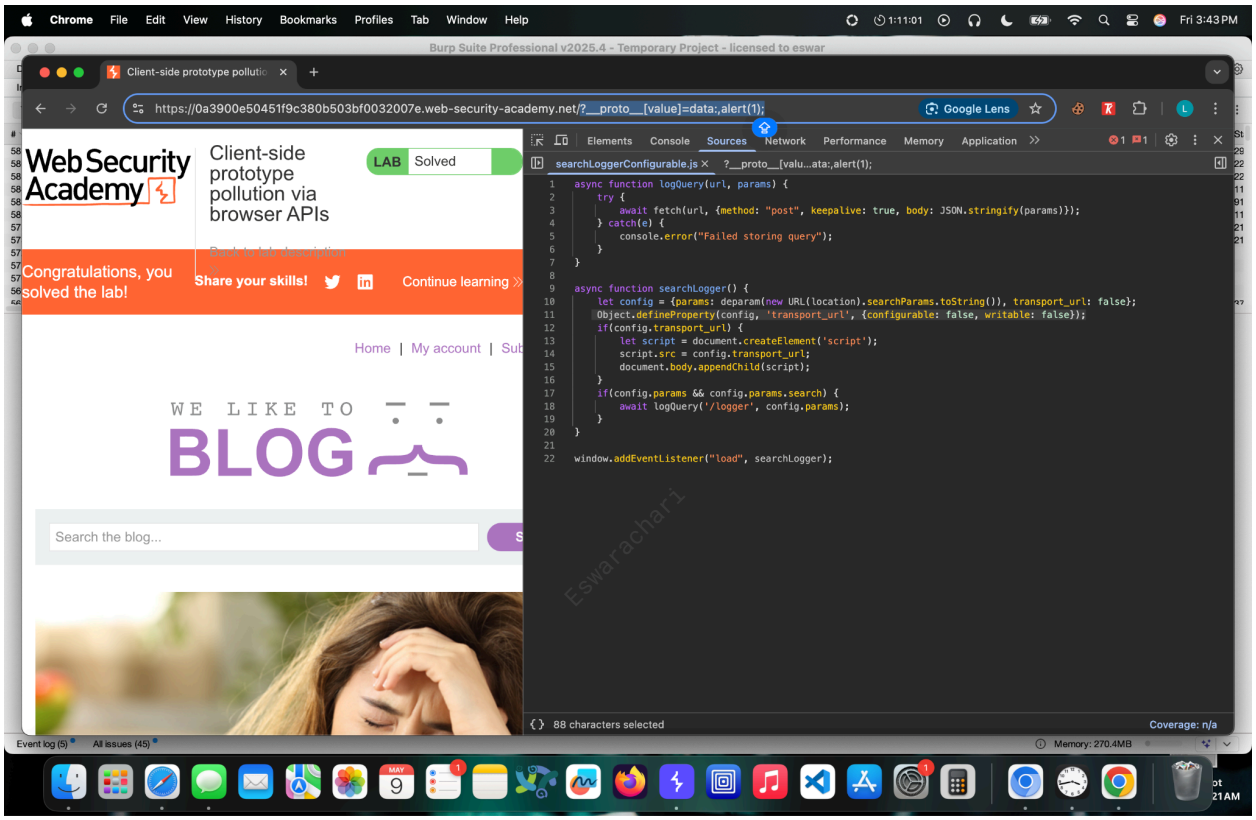
Summary: This lab illustrates the risk of prototype pollution vulnerabilities in third-party JavaScript libraries. The application uses a library that is susceptible to prototype pollution. By exploiting this vulnerability, I was able to inject malicious code that affected the application's behavior. This emphasizes the importance of keeping third-party libraries up-to-date and being aware of their potential vulnerabilities.

Screenshot Context: The screenshot shows an exploit server delivering a payload that exploits the prototype pollution vulnerability. The payload redirects the user and injects `alert(document.cookie)`.

5. Client-side prototype pollution via browser APIs

Lab Name: Client-side prototype pollution via browser APIs

Vulnerability Type: Client-Side Prototype Pollution



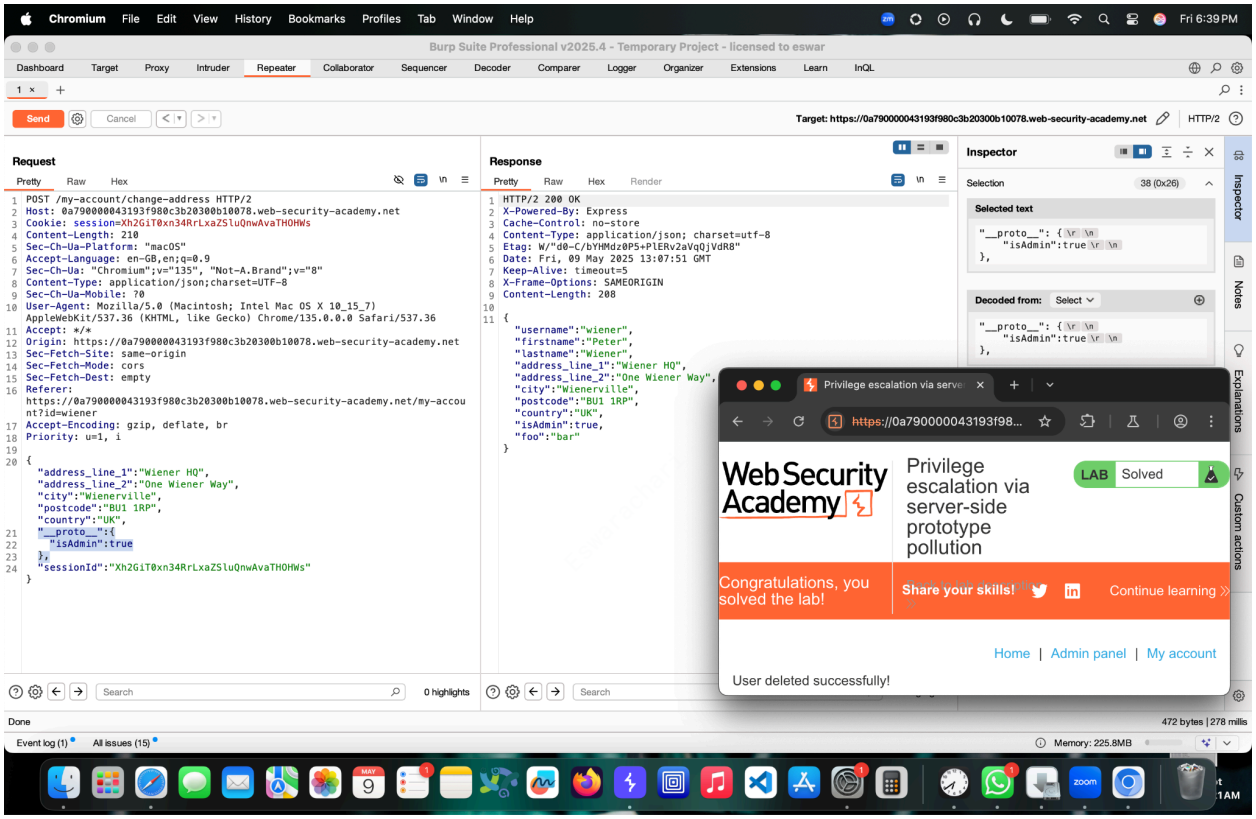
Summary: This lab demonstrates how prototype pollution can be achieved through manipulation of browser APIs. By exploiting how the application processes URL parameters, I was able to pollute the prototype and inject malicious JavaScript. This highlights that even browser APIs can be a source of prototype pollution if not handled carefully.

Screenshot Context: The screenshot shows the URL with the `_proto_[value]` parameter set to `data:,alert(1);`.

6. Privilege escalation via server-side prototype pollution

Lab Name: Privilege escalation via server-side prototype pollution

Vulnerability Type: Server-Side Prototype Pollution



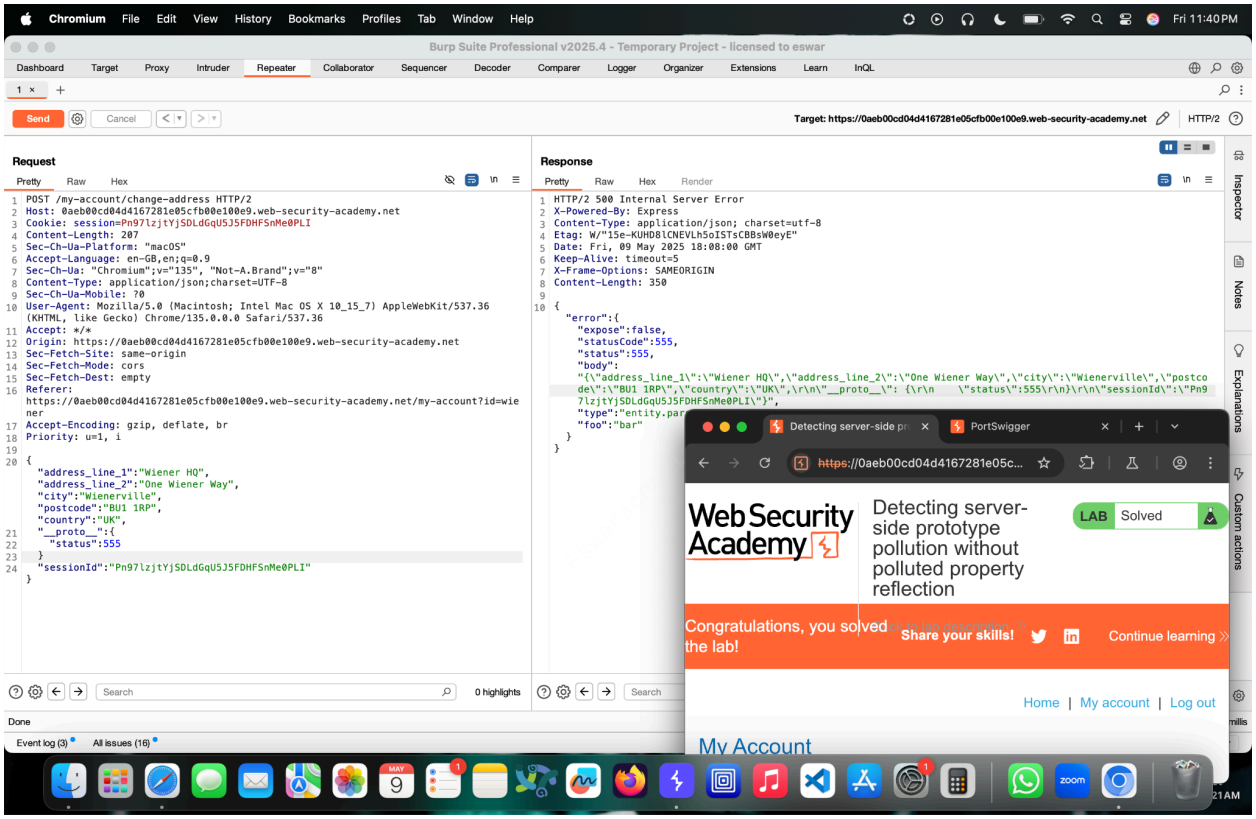
Summary: This lab demonstrates how server-side prototype pollution can lead to privilege escalation. By injecting a payload that modifies the prototype of an object on the server-side, I was able to set the `"isAdmin"` property to "true," granting myself administrative privileges. This highlights the severe consequences of server-side prototype pollution, potentially allowing attackers to gain full control of the application.

Screenshot Context: The screenshot shows the Burp Suite request with the `_proto_` payload that injects the `"isAdmin":true` property.

7. Detecting server-side prototype pollution without polluted property reflection

Lab Name: Detecting server-side prototype pollution without polluted property reflection

Vulnerability Type: Server-Side Prototype Pollution



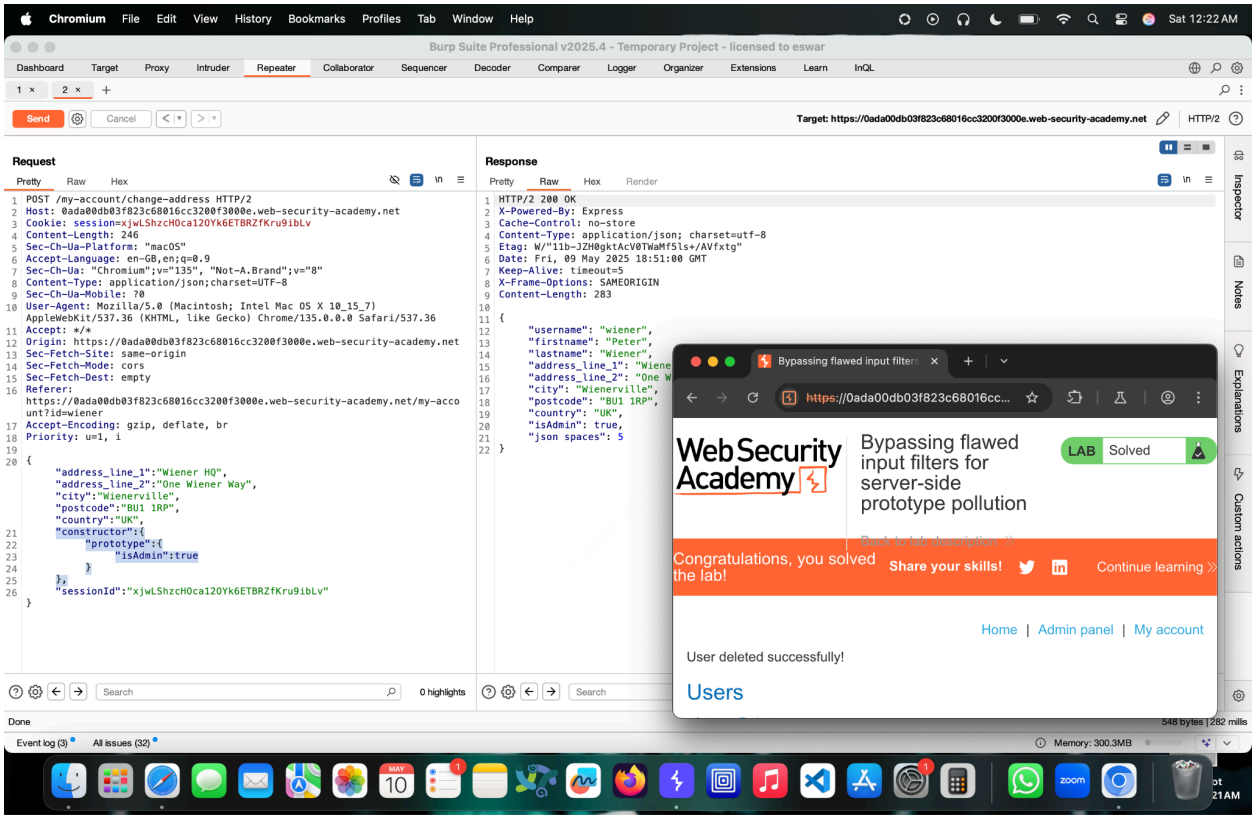
Summary: This lab focuses on detecting server-side prototype pollution in situations where the polluted property is not directly reflected in the response. I exploited the vulnerability to cause a server-side error (HTTP 500 Internal Server Error) by polluting the `status` property of the `_proto_` object. This demonstrates techniques for identifying prototype pollution even when there is no direct output of the polluted value.

Screenshot Context: The screenshot shows the Burp Suite request with the `_proto_` payload that sets the "status" property and the server's 500 Internal Server Error response.

8. Bypassing flawed input filters for server-side prototype pollution

Lab Name: Bypassing flawed input filters for server-side prototype pollution

Vulnerability Type: Server-Side Prototype Pollution



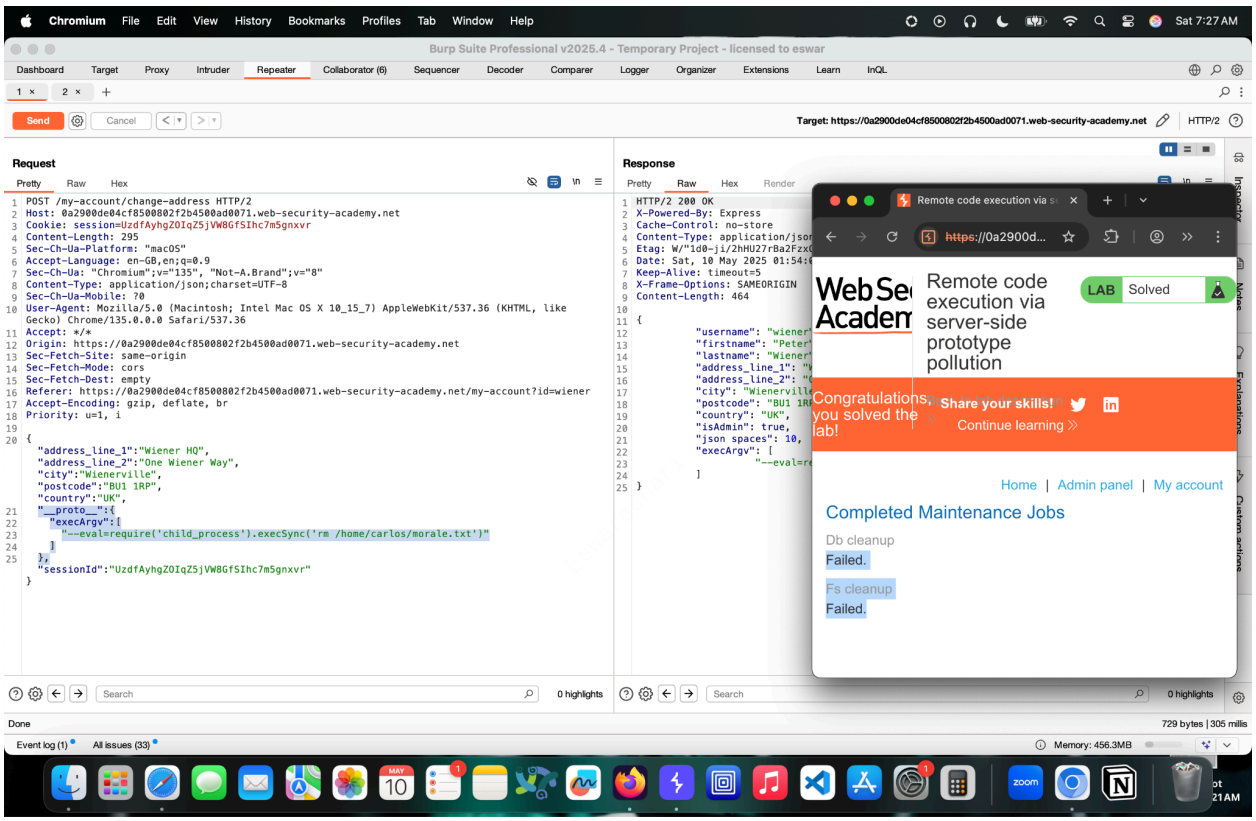
Summary: This lab involves bypassing input filters to exploit a server-side prototype pollution vulnerability. The application attempts to prevent prototype pollution by filtering certain keywords, but I was able to circumvent these filters using alternative syntax (`constructor.prototype`). By successfully polluting the prototype, I could modify application behavior. This lab emphasizes the importance of robust input validation and the potential for bypasses.

Screenshot Context: The screenshot shows the Burp Suite request with the `constructor.prototype` payload used to bypass the input filters

9. Remote code execution via server-side prototype pollution

Lab Name: Remote code execution via server-side prototype pollution

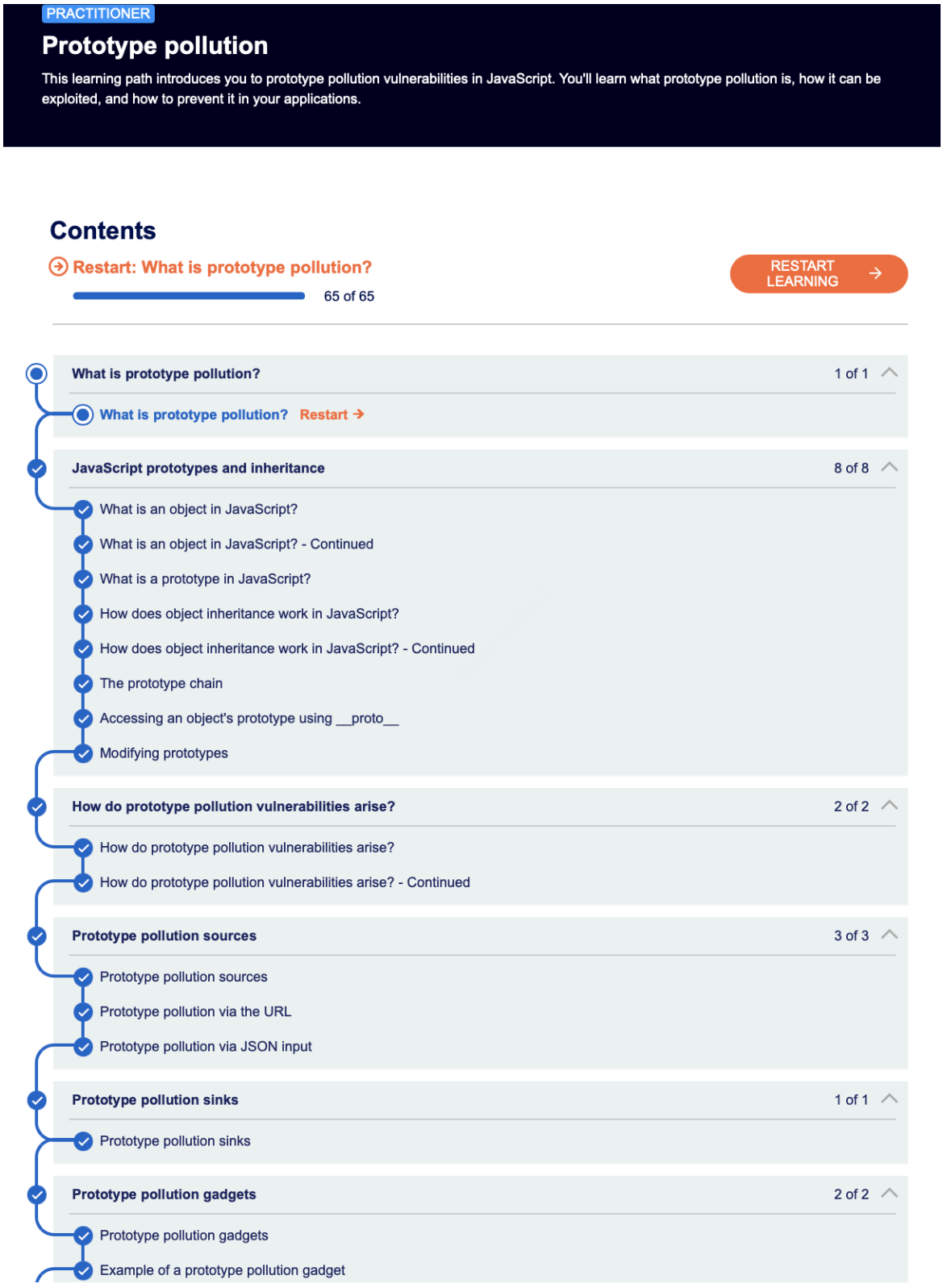
Vulnerability Type: Server-Side Prototype Pollution, Remote Code Execution (RCE)



Summary: This lab demonstrates a critical vulnerability where server-side prototype pollution can be escalated to remote code execution (RCE). By carefully crafting a payload that exploits how the server processes certain properties (in this case, `execArgv`), I was able to inject and execute arbitrary commands on the server. This lab highlights the extreme danger of server-side prototype pollution, as it can allow attackers to completely compromise the server.

Screenshot Context: The screenshot shows the Burp Suite request with the `_proto_` payload containing the `execArgv` property used to inject the RCE payload. The payload executes the command `rm /home/carlos/morale.txt`

Prototype Pollution: Key Topics Explored:



Client-side prototype pollution — 11 of 11

- Finding client-side prototype pollution sources manually
- Finding client-side prototype pollution sources using DOM Invader
- Finding client-side prototype pollution gadgets manually
- Finding client-side prototype pollution gadgets using DOM Invader
- Lab: DOM XSS via client-side prototype pollution PRACTITIONER
- Lab: DOM XSS via an alternative prototype pollution vector PRACTITIONER
- Prototype pollution via the constructor
- Bypassing flawed key sanitization
- Lab: Client-side prototype pollution via flawed sanitization PRACTITIONER
- Prototype pollution in external libraries
- Lab: Client-side prototype pollution in third-party libraries PRACTITIONER

Prototype pollution via browser APIs — 5 of 5

- Prototype pollution via browser APIs
- Prototype pollution via fetch()
- Prototype pollution via fetch() - Continued
- Prototype pollution via Object.defineProperty()
- Lab: Client-side prototype pollution via browser APIs PRACTITIONER

Server-side prototype pollution — 27 of 27

- Server-side prototype pollution
- Why is server-side prototype pollution more difficult to detect?
- Detecting server-side prototype pollution via polluted property reflection
- Detecting server-side prototype pollution via polluted property reflection - Continued
- Detecting server-side prototype pollution via polluted property reflection - Continued
- Lab: Privilege escalation via server-side prototype pollution PRACTITIONER
- Detecting server-side prototype pollution without polluted property reflection
- Status code override
- Status code override - Continued
- JSON spaces override
- JSON spaces override - Continued
- Charset override
- Charset override - Continued
- Charset override - Continued
- Lab: Detecting server-side prototype pollution without polluted property reflection PRACTITIONER
- Scanning for server-side prototype pollution sources

- Lab: Detecting server-side prototype pollution without polluted property reflection PRACTITIONER
- Scanning for server-side prototype pollution sources
- Scanning for server-side prototype pollution sources - Continued
- Bypassing input filters for server-side prototype pollution
- Lab: Bypassing flawed input filters for server-side prototype pollution PRACTITIONER
- Remote code execution via server-side prototype pollution
- Identifying a vulnerable request
- Identifying a vulnerable request - Continued
- Remote code execution via child_process.fork()
- Lab: Remote code execution via server-side prototype pollution PRACTITIONER
- Remote code execution via child_process.execSync()
- Remote code execution via child_process.execSync() - Continued
- Remote code execution via child_process.execSync() - Continued

Preventing prototype pollution 5 of 5

- Preventing prototype pollution vulnerabilities
- Sanitizing property keys
- Preventing changes to prototype objects
- Preventing an object from inheriting properties
- Using safer alternatives where possible